My Report

Nicolas Dassas-Henny, Dionysis Yusofy, Mees Kees

Saturday 14th March, 2026

Contents

1 An Introduction to Haskellog **2**

2 basic types **2**

3 ruleParser **3**

4 Unify **4**

5 Terminal User Interface **5**

5.1 Rendering 6

5.2 Event handling 6

5.3 App definition 7

5.4 Entry point 7

Bibliography **7**

1 An Introduction to Haskellog

Haskellog is an interpreter for a logic programming language similar to Prolog (with a few functionalities missing and a slightly different syntax). The user gives a semantic meaning to some functions, and the program can then answer queries: it can determine whether a certain formula can be derived from the specified rules and, if it can, provide an instantiation of the variables that makes it derivable.

The program expects a file containing logical rules made from terms. Terms are made with functions in lower case, variables in upper case, constants, which are simply functions of arity 0 and integers which are just syntactic sugar for Peano arithmetic: $0 := z()$, $1 := s(0)$, $2 := s(s(0))$, ... Each rule is either a fact, which is a single term like `plus(X,z,X).` that you could interpret as $\forall X \in \mathbb{N}, X + 0 = X$ or a rule composed of a conclusion and some premises, such as `sum_of_consecutives(N) :- plus(A, 1, B), plus(A,B,N).` which could be interpreted as "a number N is a sum of two consecutive numbers A and B if $A + B = N$ and $A + 1 = B$". Obviously these interpretations are only given by the rules: the program does not know what `plus` means to you and if you define it, it is your responsability to provide rules that match your intended meaning of each predicate.
For example, we can enrich the semantics of `plus` that we started defining before so that it matches the idea that `plus(X,Y,Z)` should be derivable if $X + Y = Z$ by adding the rule `plus(X,s(Y),s(Z)) :- plus(X,Y,Z).`.

The program can then answer queries, which are a succession of terms interpreted as a conjunction. Continuing with the previous example, a query could be `?- plus(X,Y,4), plus(X,2,Y)` which receives a positive answer from the program: `yes, X = 1 and Y = 3`. Once Haskellog has output one possible answer, the user can ask for additional ones.
You might think that the query `?- plus(X,Y,4), plus(X,1,Y)` would be given a similar answer `yes, X = 1,5 and Y = 2,5` but remember that we never defined what 1,5 or 2,5 are, the semantics only talks about integers. Therefore, the answer would be `no`. Try implementing half-integers to change that!

Internally, Haskellog works as follows: a parser reads the source file and transforms the rules and queries in some Haskell data types. Then, when a query is inputed in the UI, it is parsed and the prover attempts to solve it by resolution: the query is viewed as a conjunction of goals, and Haskellog repeatedly selects one goal and tries to match it with the head of a fact or rule from the input file. This matching is performed by unification, which computes a substitution for variables making two terms identical when possible. If no such substitution exists, that branch of the search fails and the program backtrack. When a rule matches, its premises replace the resolved goal, and the process continues recursively until either all goals have been solved, yielding a successful proof and an answer substitution, or no rule applies. The result of the proof is then presented to the user.

2 basic types

This module defines our basic types

```haskell
module Terms where

import Data.Map (Map)
```

```
type Variable = String -- A variable is just defined by its name

data Term = Var Variable | Fun String [Term] -- A term is either a variable or a function (
    a 0-ary function being a constant)
  deriving (Eq,Ord,Show)

data Clause = Fact Term | Rule Term [Term] -- A Clause is either an axiom or a conclusion
    and a set of premisses

type Program = [Clause] -- A program is a set of clauses

type Query = [Term] -- A query is a set of terms, interpreted as a conjunction

type Subst = Map Variable Term -- A substitution associates a term to a variable
```

3 ruleParser

In This section we look at the parser for the rules. The rules should be fed as as a regular text file. Here some examples for rules:

```
bigger(cat(), mouse()):-
bigger(dog(), cat()):-
bigger(horse(), dog()):-
is_bigger(A,B):- bigger(A,B)
is_bigger(A,B):- bigger(A,Z) & is_bigger(Z,B)
```

We interpret constants as 0-ary functions. The ":-" can be read as "if". We only allow horn-clauses so the only connector is conjunction for now.
We use Parsec for the parsing. Now to parse the rules we first establish some seperators:

```
sepConcAssum :: Parsec String () ()
sepConcAssum = do
                spaces
                string ":-"
                spaces

andSep :: Parsec String () ()
andSep = do
          spaces
          char '&'
          spaces

termSep :: Parsec String () ()
termSep = do
          spaces
          char ','
          spaces

endOfList :: Parsec String () ()
endOfList = do
          spaces
          char ')'
          spaces
```

Now every Rule is a conclusion and a list of assumptions. We first seperate the clause in its two parts:

```
pRule :: Parsec String () Rule
pRule = do
          conc<- pConc
          sepConcAssum
```

```
          assums <- pAssums
          return (conc, assums)
```

Where the conclusion is just parsed as a term and the assumption as a list of terms.
We still have to implement that the list of assumption can be empty.

```
pConc :: Parsec String () Conclusion
pConc = pTerm

pAssums :: Parsec String () [Assumption]
pAssums = many $ do
    assum <- pTerm
    eof <|> andSep
    return assum
```

We allready made it possible in for the list of terms in a funciton to be empty.

```
pTerms :: Parsec String () [Term]
pTerms = list <|> emptyList where
    emptyList = spaces >> char ')' >> return []
    list = many $ do
      assum <- pTerm
      endOfList <|> termSep
      return assum
```

Note that the "try lookAhead" is checking the next character without "devouring it". So we can check if the first letter is capatalized or not. This is our convention for funcitons vs variable.

```
pTerm :: Parsec String () Term
pTerm = pVar <|> pFunc where
        pVar = do
              (try $ lookAhead $ oneOf ['A'.. 'Z'])
              V . read <$> many1 anyChar
        pFunc = do
              (try $ lookAhead $ oneOf ['a'.. 'z'])
              a <- (read <$> manyTill anyChar (oneOf " ("))
              b <- pTerms
              return (Fun a b)
```

What is still left to do is to check for spaces at the right places.

4 Unify

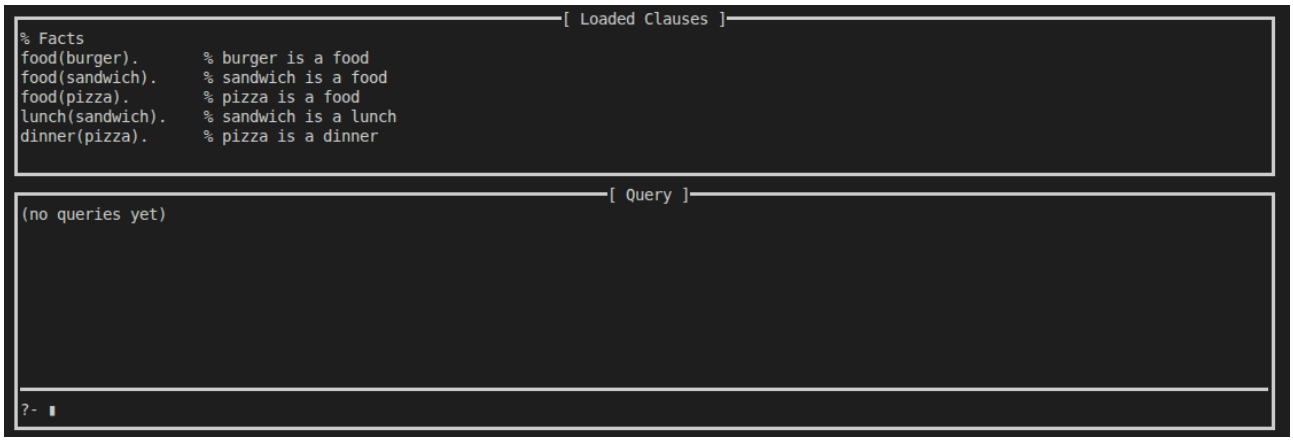
This module implement unification.

```
module Unify where

import qualified Data.Map as Map
import Terms

applySubst :: Subst -> Term -> Term -- substitute every variable in a term with their
    associated value
applySubst s (Var x) =
  case Map.lookup x s of
    Nothing -> Var x
    Just t -> applySubst s t -- we propagate the substitution
applySubst s (Fun name l) =
  Fun name (map (applySubst s) l)

occurrenceCheck :: String -> Term -> Bool -- check if the name of a variable appears in a
    term
occurrenceCheck x (Var y) = x == y
occurrenceCheck x (Fun _ l) = any (occurrenceCheck x) l
```

```haskell
unification :: Subst -> Term -> Term -> Maybe Subst
-- return a substitution if it finds one that unify two terms under a predefined
    substitution.
unification sub t1 t2 =
  case (applySubst sub t1, applySubst sub t2) of
    (Var x, Var y) ->
      if x == y then Just sub else Just (Map.insert x (Var y) sub)
    (Var x, fun) ->
      if occurrenceCheck x fun then Nothing else -- The variable appears in the function
        Just (Map.insert x fun sub)
    (fun, Var y) ->
      if occurrenceCheck y fun then Nothing else -- The variable appears in the function
        Just (Map.insert y fun sub)
    (Fun name1 l1, Fun name2 l2)
      | name1 /= name2 -> Nothing
      | otherwise -> unifyLists sub l1 l2

unifyLists :: Subst -> [Term] -> [Term] -> Maybe Subst
unifyLists sub [] [] = Just sub
unifyLists sub (x1:xs1) (x2:xs2) =
  unification sub x1 x2 >>= (\ s -> unifyLists s xs1 xs2)
unifyLists _ _ _ = Nothing
```

5 Terminal User Interface

This is our main way to interact with the project. The project can be compiled using `stack build` and then launched with `stack exec myprogram`. Arguments can be given to the program to load prolog rules and facts into the program. This can be done for example as such: `stack exec myprogram -- examples/food.pl examples/meal_rules.pl`. After running that command the user would see something like this:



Users can type prolog clauses into the query chat box. For now the functionality is quite basic as it would only return the data structure it parsed and not actually return a valid prolog result. If a user would for example enter `food(X).` the terminal then would simply return `[Fun "food" [V "X"]]`

We use a library called brick to handle the UI for us. We use two named viewports so that brick can track their scroll positions independently.

```haskell
data Name = ClausesVP | HistoryVP
  deriving (Eq, Ord, Show)
```

The state of our TUI: the loaded file contents, the query history, and whatever the user is

currently typing.

```
data TuiState = TuiState
  { clausesText :: String
  , history     :: [String]
  , inputBuf    :: String
  }
```

5.1 Rendering

We draw two bordered panes stacked on top of each other.

```
drawClausesPane :: TuiState -> Widget Name
drawClausesPane s =
  let content = if null (clausesText s)
                then str "(no file loaded)"
                else str (clausesText s)
  in withBorderStyle unicodeBold
     (borderWithLabel (str "[ Loaded Clauses ]")
     (viewport ClausesVP Vertical content))

drawQueryPane :: TuiState -> Widget Name
drawQueryPane s =
  let histWidgets = if null (history s)
                    then [str "(no queries yet)"]
                    else map str (history s)
      inputLine   = str "?- " <+> str (inputBuf s) <+> str "\9646"
  in withBorderStyle unicodeBold
     (borderWithLabel (str "[ Query ]")
     (vBox [viewport HistoryVP Vertical (vBox histWidgets), hBorder, inputLine]))

drawUI :: TuiState -> [Widget Name]
drawUI s = [vBox [vLimitPercent 40 (drawClausesPane s), drawQueryPane s]]
```

5.2 Event handling

The event handling handles how the program should behave on signal input. This manages basic chatbox functionality such as typing or deleting characters. It also handles when to close the program with ctrl+c or by pressing the esc character.

```
addDot :: String -> String
addDot q = if null q || last q == '.' then q else q ++ "."

handleEvent :: BrickEvent Name () -> EventM Name TuiState ()

-- quit on Esc or Ctrl-C
handleEvent (VtyEvent (V.EvKey V.KEsc [])) = halt
handleEvent (VtyEvent (V.EvKey (V.KChar 'c') [V.MCtrl])) = halt

-- submit query on Enter: parse and show the resulting [Term]
handleEvent (VtyEvent (V.EvKey V.KEnter [])) = do
  s <- get
  let q      = addDot (inputBuf s)
      result = case parseQuery q of
                 Left err    -> "Parse error: " ++ show err
                 Right terms -> show terms
      newLines = ["?- " ++ q, result, ""]
  modify (\st -> st { history = history st ++ newLines, inputBuf = "" })
  vScrollToEnd (viewportScroll HistoryVP)

-- backspace to delete character
handleEvent (VtyEvent (V.EvKey V.KBS [])) = do
```

```
  s <- get
  let buf = inputBuf s
  if null buf
    then return ()
    else modify (\st -> st { inputBuf = init buf })

-- typing characters
handleEvent (VtyEvent (V.EvKey (V.KChar c) [])) =
  modify (\s -> s { inputBuf = inputBuf s ++ [c] })

-- scroll the clauses pane
handleEvent (VtyEvent (V.EvKey V.KUp [])) =
  vScrollBy (viewportScroll ClausesVP) (-1)
handleEvent (VtyEvent (V.EvKey V.KDown [])) =
  vScrollBy (viewportScroll ClausesVP) 1

-- ignore everything else
handleEvent _ = return ()
```

5.3 App definition

The app definition bundles together the drawing, event handling, cursor, and styling logic into a single Brick application value.

```
tuiApp :: App TuiState () Name
tuiApp = App
  { appDraw         = drawUI
  , appChooseCursor = showFirstCursor
  , appHandleEvent  = handleEvent
  , appStartEvent   = return ()
  , appAttrMap      = const (attrMap V.defAttr [])
  }
```

5.4 Entry point

This code is run by the main function in Main.lhs. It initializes the TuiState with the loaded clauses and an empty history and input buffer, then runs the Brick application.

```
runTui :: String -> IO ()
runTui clauses = do
  let initState = TuiState
        { clausesText = clauses
        , history     = []
        , inputBuf    = ""
        }
  _ <- defaultMain tuiApp initState
  return ()
```

References